

SIH26151 â€” Branch Execution Runbook

How humans and agents must work

- Execute exactly one branch at a time.
- Read `SIH26151\_FINAL\_BUILD\_GUIDE.md`, `SIH26151\_APP\_DATA\_FLOW\_FINAL.md` and `SIH26151\_EDGE\_CASE\_REGISTER.md` before each branch.
- Do not change open-source upstream code. Build local adapters only.
- Do not replace baseline modules without an approved architecture decision record.
- Do not enable live collection by default. Fixture replay is the default mode.

Standard branch procedure

1. Create branch: `feat/branch-<n>-<short-name>`.
2. Copy relevant acceptance criteria and edge-case IDs into PR description/task.
3. Add/update fixture before implementation.
4. Implement only branch scope.
5. Add unit tests and one end-to-end integration test for the branchâ€™s data path.
6. Run `docker compose up --build`, tests, lint, dependency/license check and secret scan.
7. Produce acceptance proof: API example, automated test output, screenshot or report artifact.
8. Update docs/contracts/third-party notices.
9. Review against stop-the-line criteria.
10. Merge only after acceptance passes.

Branch commands template

```
```text
Start local services
cp .env.example .env
make up

Run tests
make test
make lint

Reset deterministic demo data
make demo-reset

Stop services
make down
```
```

If the current repository does not have these commands, add them in Branch 0/1 without changing product behavior.

Branch 0 instructions

Deliver `docs/baseline-audit.md` only. Do not modify runtime code. For each claimed existing module record: path, entry point, dependency list, input format, output format, current test evidence, security concerns, data-provenance fields missing and exact adapter required.

Branch 1 instructions

Create canonical schemas and local adapters. Add schema tests first. Legacy modules retain their logic; adapter converts output. Fusion accepts new evidence units and may use compatibility input until all adapters are migrated.

Branch 2 instructions

Add Capture, source policy, provenance and safe statuses. Test fixture replay before any authorized network path. Ensure a failed source produces a durable status record, not deleted evidence.

Branch 3 instructions

Implement `config/scoring.yaml`, noisy-OR, score caps, tiers, conflict sets, deterministic explanations and link versioning. Test duplicate, stale, negative, conflicting and boundary cases.

Branch 4 instructions

Package OnionScan as a pinned external executable/container. It receives only allowlisted targets. Execute non-root with hard timeout, output limit and no access beyond its required proxy/network route. Store raw output hash; adapter maps only known fields.

Branch 5 instructions

Extend existing dashboard. The first feature is the edge evidence drawerâ€”not graph beautification. Every action must preserve role checks and create audit events.

Branch 6 instructions

Use three time fields consistently: source-claimed time, observation time and capture time. Display uncertainty. Never invent an event date from an untrusted timestamp.

Branch 7 instructions

Build behavior evidence only from cleaned, chronologically ordered posts. Require non-name contextual signal. A migration candidate is an analyst lead, never a merge.

Branch 8 instructions

Classical stylometry is optional. If time is limited, do not create a rushed weak implementation; retain MiniLM with stated limitations. If built, enforce corpus/language gates and create adversarial/boilerplate fixtures.

Branch 9 instructions

Make reports snapshots. Case export stores a manifest of exact evidence IDs, link versions, decisions and model configuration. Do not query "latest state" while rendering a report.

Branch 10 instructions

Keep PostgreSQL canonical. Neo4j is a projection. Dual-write temporarily and reconcile counts/IDs/evidence references before switching graph reads.

Branch 11 instructions

Demo must work with no internet. Use synthetic non-illegal fixture data. Rehearse easy, hard, rebrand and negative cases. The negative case is mandatory: it proves the system limits false positives.

Agent prompt template

```text

You are implementing Branch <NUMBER> only of SIH26151. Treat the Final Build Guide, App/Data Flow specification and Edge-Case Register as authoritative.

Before coding, list: branch scope, excluded scope, affected files, data-contract changes, applicable edge-case IDs, tests/fixtures to add, rollback approach and acceptance criteria.

Do not modify upstream open-source code. Use adapters. Do not enable live collection. Do not produce real-world identity claims. Every new signal must output a validated EvidenceUnit with provenance, independence group and limitations.

After coding, provide exact commands to test, expected output, and a brief mapping of implementation to acceptance criteria. Stop if the repository baseline contradicts the

guide; report the contradiction instead of guessing.